

```
1: #include <stdio.h>
2: #include <stdlib.h>
3: #include <string.h>
4: #include <ctype.h>
5: #include <math.h>
6:
7: // Funci3n para encontrar la posici3n de un operador fuera de par3ntesis
8: int find_operator(const char *expr, int start, int end, const char *operators) {
9:     int parentheses_count = 0;
10:
11:     for (int i = end; i >= start; i--) {
12:         if (expr[i] == ')') parentheses_count++;
13:         else if (expr[i] == '(') parentheses_count--;
14:         else if (parentheses_count == 0) {
15:             // Buscar el operador en la cadena de operadores
16:             for (int j = 0; operators[j] != '\0'; j++) {
17:                 if (expr[i] == operators[j]) {
18:                     return i;
19:                 }
20:             }
21:         }
22:     }
23:     return -1;
24: }
25:
26: // Funci3n para evaluar una expresi3n simple (sin operadores)
27: double evaluate_simple(double a, double b, char op) {
28:     switch(op) {
29:         case '+': return a + b;
30:         case '-': return a - b;
31:         case '*': return a * b;
32:         case '/':
33:             if (b != 0) return a / b;
34:             else {
35:                 printf("Error: Divisi3n por cero\n");
36:                 exit(1);
37:             }
38:         case '^': return pow(a, b);
39:         default: return 0;
40:     }
41: }
42:
43: // Funci3n recursiva principal
44: double evaluate_expression(const char *expr, int start, int end) {
45:     // Eliminar espacios
46:     while (start <= end && isspace(expr[start])) start++;
47:     while (end >= start && isspace(expr[end])) end--;
48:
49:     if (start > end) return 0;
50:
51:     // Caso base: expresi3n entre par3ntesis
52:     if (expr[start] == '(' && expr[end] == ')') {
53:         // Verificar que sea un par3ntesis que englobe toda la expresi3n
54:         int balance = 0;
55:         int is_balanced = 1;
56:         for (int i = start; i <= end; i++) {
57:             if (expr[i] == '(') balance++;
58:             else if (expr[i] == ')') balance--;
59:             if (balance == 0 && i < end) {
60:                 is_balanced = 0;
61:             }
62:         }
63:         if (is_balanced) {
64:             return evaluate_expression(expr, start + 1, end - 1);
65:         }
66:     }
67:
68:     // Buscar operadores por orden de precedencia (de menor a mayor)
69:     int pos;
```

```
71:
72: // Suma y resta (menor precedencia)
73: pos = find_operator(expr, start, end, "+-");
74: if (pos != -1) {
75:     double left = evaluate_expression(expr, start, pos - 1);
76:     double right = evaluate_expression(expr, pos + 1, end);
77:     return evaluate_simple(left, right, expr[pos]);
78: }
79:
80: // Multiplicaci3n y divisi3n
81: pos = find_operator(expr, start, end, "*");
82: if (pos != -1) {
83:     double left = evaluate_expression(expr, start, pos - 1);
84:     double right = evaluate_expression(expr, pos + 1, end);
85:     return evaluate_simple(left, right, expr[pos]);
86: }
87:
88: // Potencia (mayor precedencia)
89: pos = find_operator(expr, start, end, "^");
90: if (pos != -1) {
91:     double left = evaluate_expression(expr, start, pos - 1);
92:     double right = evaluate_expression(expr, pos + 1, end);
93:     return evaluate_simple(left, right, expr[pos]);
94: }
95:
96: // Caso base: n3mero
97: char num_str[100];
98: int j = 0;
99: for (int i = start; i <= end; i++) {
100:     if (isdigit(expr[i]) || expr[i] == '.' || (i == start && expr[i] == '-'))
{
101:         num_str[j++] = expr[i];
102:     }
103: }
104: num_str[j] = '\0';
105: return atof(num_str);
106: }
107:
108: // Funci3n wrapper
109: double evaluate(const char *expression) {
110:     return evaluate_expression(expression, 0, strlen(expression) - 1);
111: }
112:
113: // Funci3n principal con ejemplos
114: int main() {
115:     char expression[200];
116:
117:     printf("Calculadora recursiva de expresiones aritm3ticas\n");
118:     printf("Operadores soportados: +, -, *, /, ^\n");
119:     printf("Soporta par3ntesis anidados\n");
120:     printf("Ejemplo: (2+3)*4, 2^3+1, ((2+3)*(4-1))\n\n");
121:
122:     while (1) {
123:         printf("Ingrese expresi3n (o 'exit' para salir): ");
124:         fgets(expression, sizeof(expression), stdin);
125:
126:         // Eliminar el salto de l3nea
127:         expression[strcspn(expression, "\n")] = '\0';
128:
129:         if (strcmp(expression, "exit") == 0) {
130:             break;
131:         }
132:
133:         double result = evaluate(expression);
134:         printf("Resultado: %.6f\n\n", result);
135:     }
136:
137:     return 0;
138: }
```